Advanced Database Systems Portfolio: ValPay

Full Name: Valentine Efagene (MIVA ID: 301921385)

Miva Email Address: v.efagene1385@miva.edu.ng

Course: MIT 8103 – Advanced Database Systems

Assessment: Continuous Assessment (September 2026 cohort) – Portfolios 1–5 and Final Documentation; Case study: ValPay

1. GITHUB REPOSITORY AND SUBMISSION

All source code, SQL and MongoDB scripts, evidence files and this document are in a public GitHub repository, so assessors can browse or clone it without requesting access: https://github.com/Valentine-Efagene/advanced-database-portfolio
The relational evidence can be regenerated with ./scripts/Efagene_Valentine_301921385_InitAndEvidence.sh, which starts PostgreSQL in Docker, loads the schema and seed data, and refreshes the text outputs in each portfolio's evidence folder. This document is also included in the submission zip as Efagene_Valentine_301921385_DOCUMENTATION.pdf.

2. SELECTED ORGANISATION

I chose ValPay, a Nigerian fintech platform offering NGN wallets, peer-to-peer (P2P) transfers, top-ups and auditable transaction history. Retail customers move money, while compliance, finance and support staff need traceability by transfer reference, ledger integrity and KYC (Know Your Customer) status. Core data includes customers, wallet accounts, double-entry ledger postings, transfer metadata, beneficiaries and audit events. The full scenario is in Efagene_Valentine_301921385_CASE_STUDY.md and is reused consistently across Portfolios 1 to 5.

3. SUMMARY OF ALL FIVE PORTFOLIOS

Portfolio 1: Database design and modelling

I modelled ValPay relationally with customer, wallet_account, wallet_transaction, ledger_posting, transfer_detail, beneficiary and transaction_audit tables. The ER diagram captures the one-to-many relationships between customers, wallets and transactions, and the ledger's dependence on accounts. The physical schema enforces non-negative balances, unique transfer references, idempotency keys and no self-transfers. In the normalisation notes I justified 1NF to 3NF: amounts are atomic, customer attributes live only on customer, and the running balance on

wallet_account is changed only inside controlled transactions rather than copied onto unrelated tables.

Portfolio 2: Query processing and optimisation

I wrote six reporting and operational queries: wallet statements, daily volume, reference lookup, low-balance alerts, failed and pending queues, and ledger reconciliation. EXPLAIN showed that Q3, which filters transactions by a reference prefix, ran a sequential scan over wallet_transaction. After I added idx_transaction_reference, the plan changed to a bitmap index scan, which is what support staff need when tracing VP-TXN references. Q1 already used idx_ledger_account on the statement join path. Query results and plans are saved in the Portfolio 2 evidence folder.

Portfolio 3: Transactions and concurrency

I implemented the P2P transfer as a single PostgreSQL transaction covering the balance check, transaction insert, paired ledger postings and audit entry. In Test A, I attempted a debit larger than the available balance: the debit update affected zero rows, the transaction rolled back, and no row remained for the failed reference, so balances were unchanged. In Test B, I committed a 2,500 NGN transfer with balanced debits and credits and correctly updated balances. I did not load-test concurrent sessions locally; in production, ValPay would add SELECT ... FOR UPDATE on wallet rows to serialise concurrent API requests.

Portfolio 4: NoSQL and advanced data models

I stored semi-structured KYC documents, activity events and fraud signals in MongoDB while keeping the ledger in PostgreSQL. KYC payloads vary by document type, so a flexible document schema avoids wide nullable SQL columns, and activity events are append-only, high-volume and queried by time range. I added JSON Schema validation, which rejects invalid enum values, and compound indexes for the pending KYC queue and time-window activity traces. Aggregation pipelines summarise fraud rule hits by device and customer. Documents link back to PostgreSQL through customer_id and the transaction reference, without copying any ledger amounts into MongoDB.

Portfolio 5: Distributed and cloud database exercise

I designed a reference production architecture for ValPay: Amazon RDS PostgreSQL in a Multi-AZ configuration with synchronous replication for ledger writes, read replicas for statements and reporting, MongoDB Atlas for KYC and events, and Redis for idempotency keys and rate limits. This design is based on vendor documentation and course material; I did not provision these cloud services for the assessment. What I ran and verified locally is the Docker Compose stack of PostgreSQL and MongoDB that produced the evidence for Portfolios 1 to 4. In CAP terms, ledger writes favour consistency over availability, while dashboards and analytics can tolerate eventually consistent reads. The write-up also outlines sharding activity_events by customer_id, partitioning ledger_posting by month at scale, and NDPA-aligned retention using TTL indexes.

4. SELECTED EVIDENCE

The figures below are screenshots from my local runs. Full text logs and further screenshots are in each portfolio's evidence folder.

Figure 1: Entity-relationship diagram for the ValPay schema (Portfolio 1)

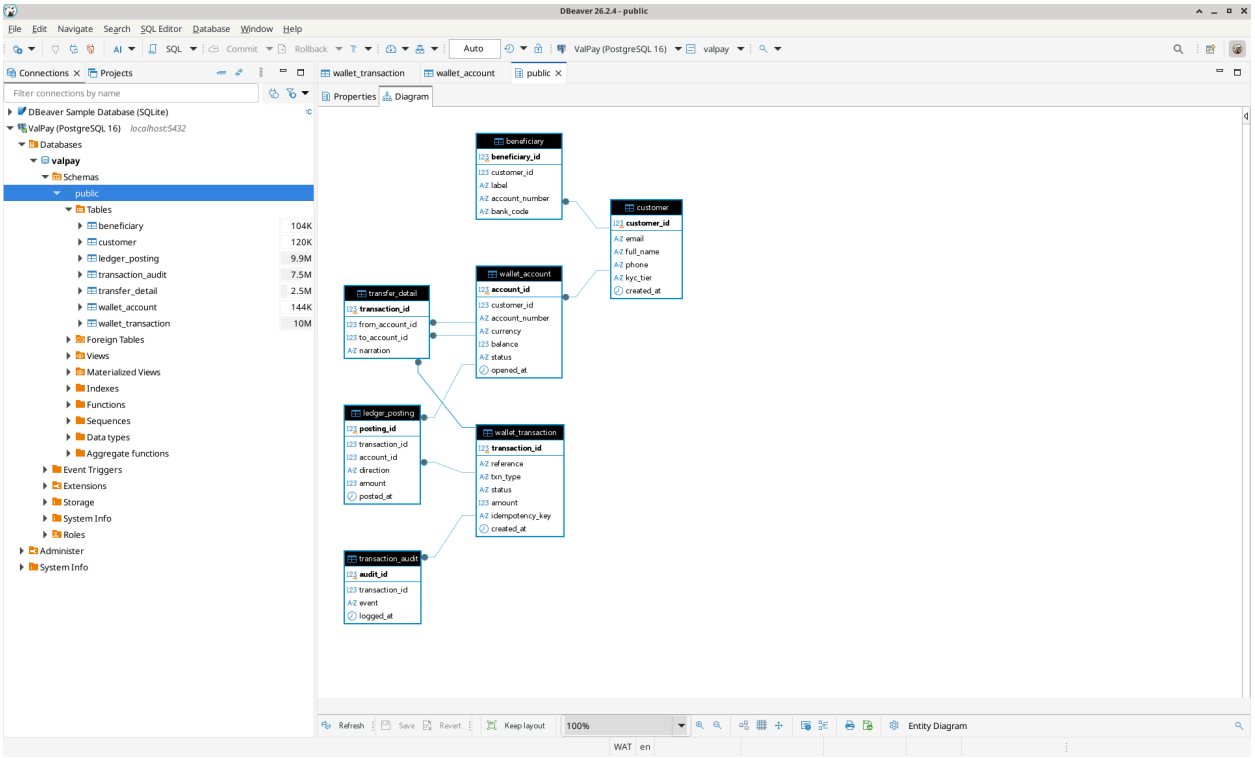


Figure 2: Q3 query plan before the index, showing a sequential scan (Portfolio 2)

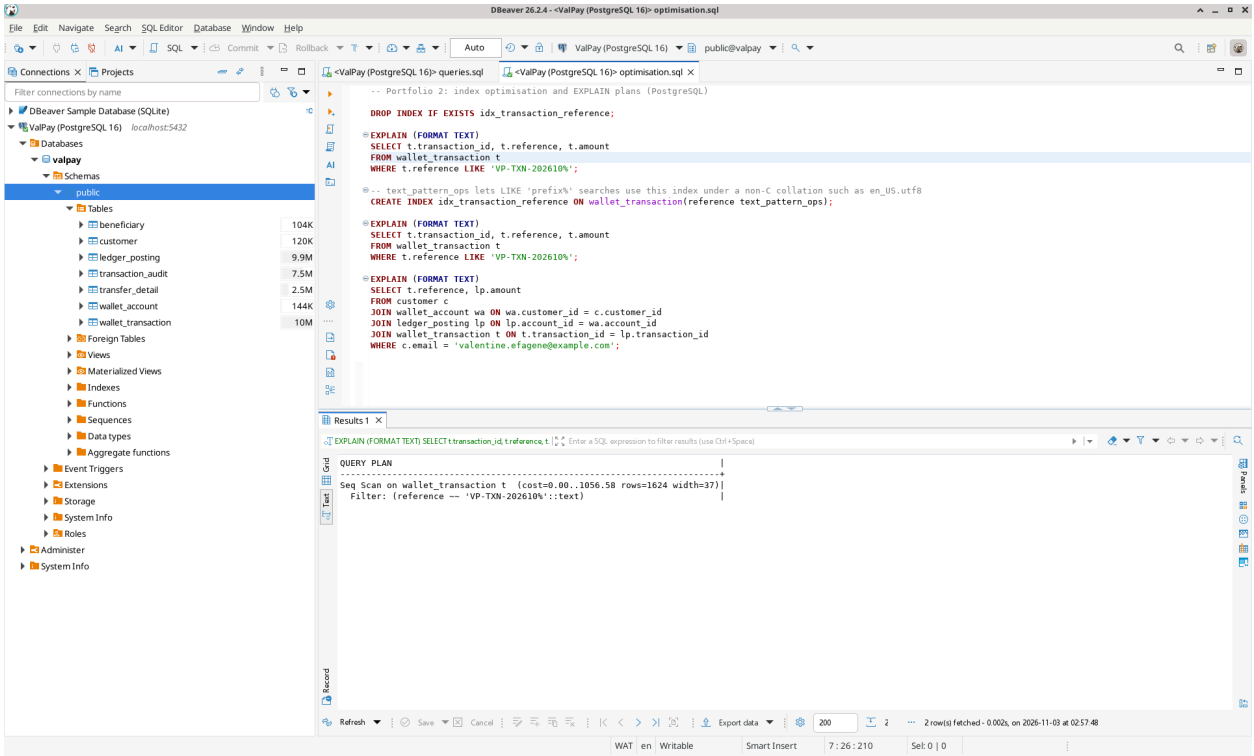


Figure 3: Q3 query plan after adding idx_transaction_reference, showing a bitmap index scan (Portfolio 2)

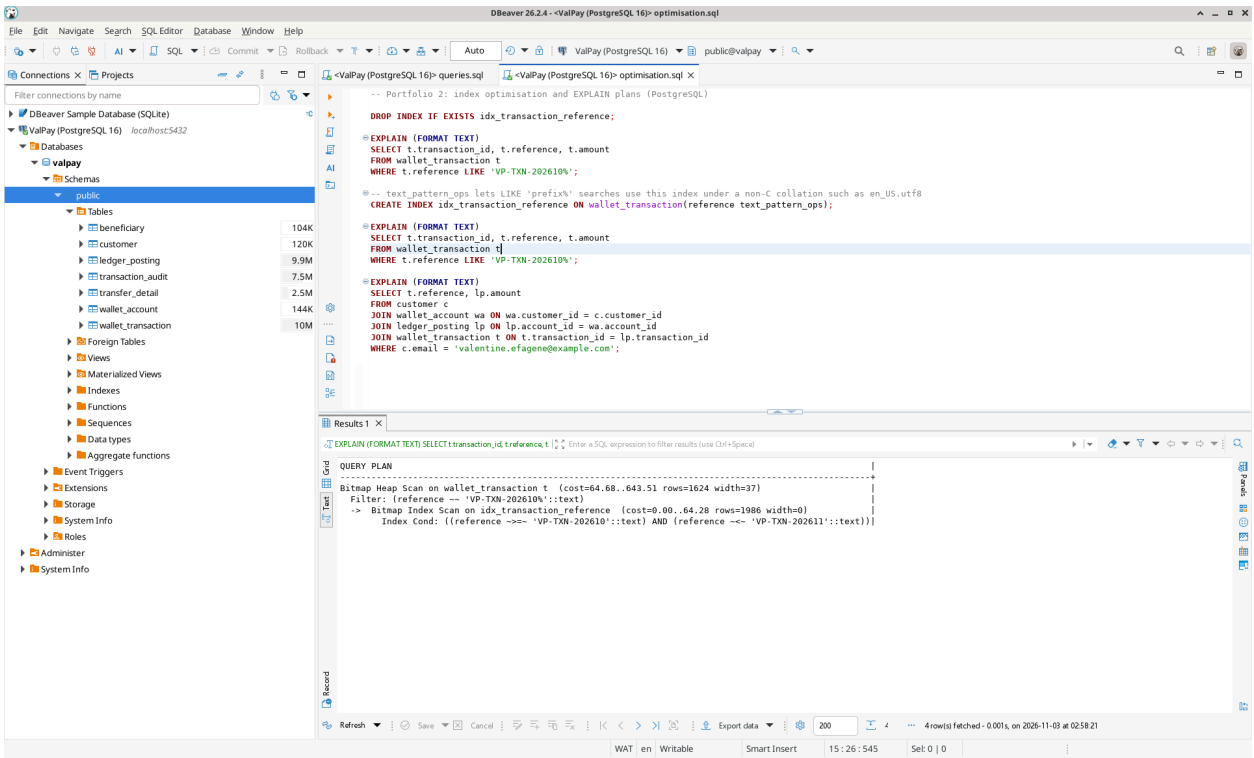


Figure 4: Insufficient-funds transfer rolled back with the sender balance unchanged (Portfolio 3)

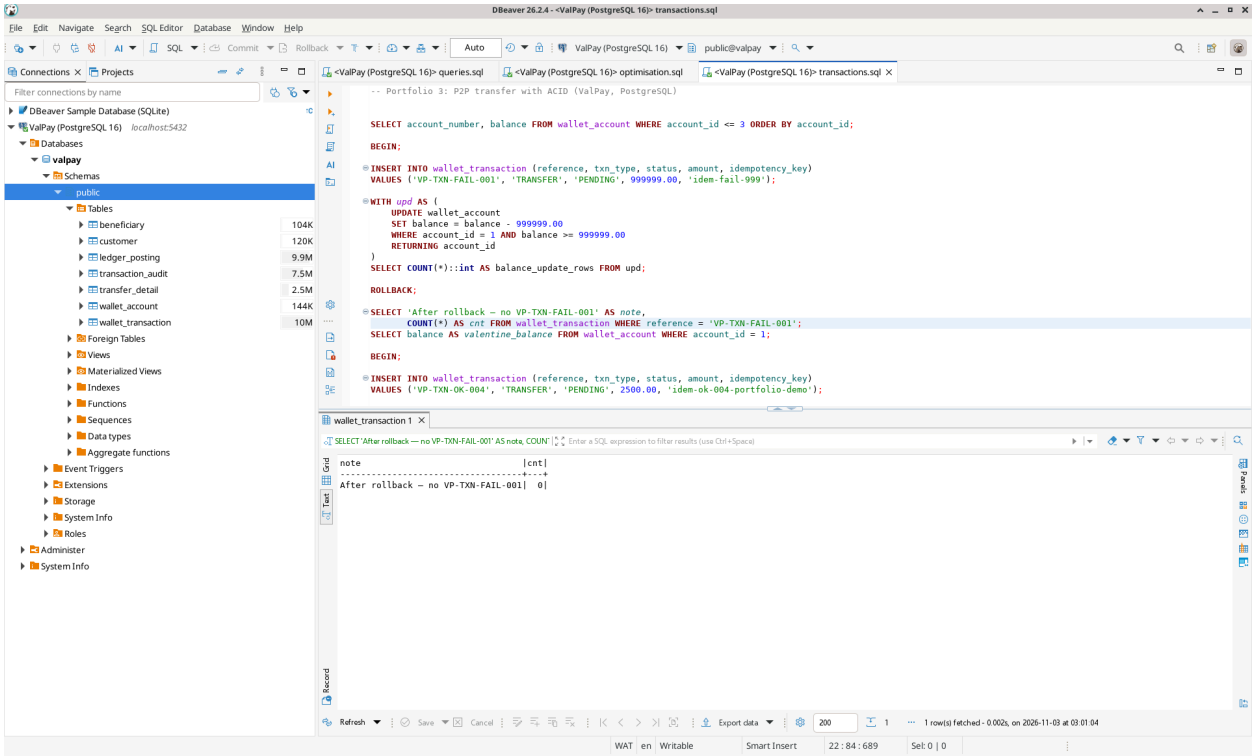
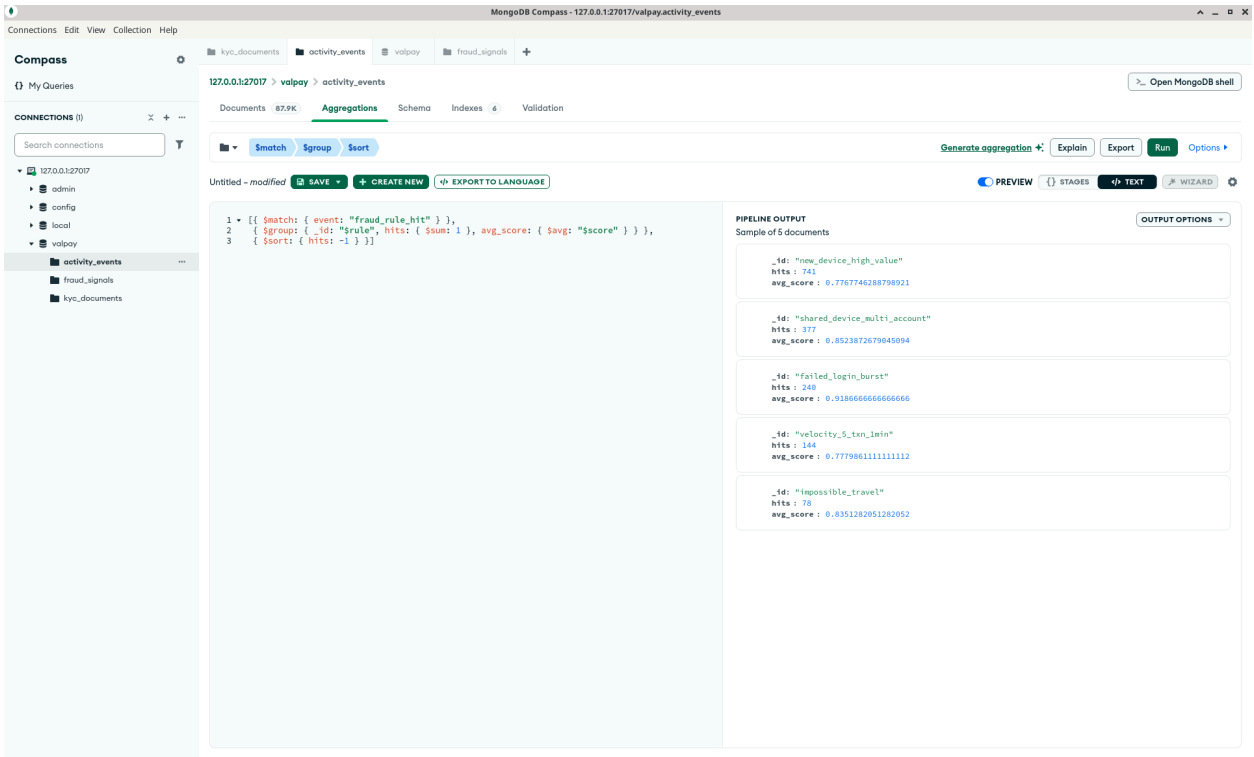


Figure 5: Fraud-signal aggregation in MongoDB Compass (Portfolio 4)



5. INDIVIDUAL REFLECTION

Design choices

I kept wallet balances and ledger postings in PostgreSQL because ValPay needs ACID guarantees and reconcilable double-entry rules. I moved variable KYC payloads, high-volume activity and fraud analytics to MongoDB, where a flexible schema and event-style storage fit better. I used surrogate keys plus unique business references (reference and idempotency_key) to mirror how payment APIs prevent duplicate debits. Storing balance on wallet_account was a deliberate read optimisation, but I only update it inside the Portfolio 3 transfer transaction so it cannot drift from the ledger.

Challenges I faced

- Normalisation versus query speed: I wanted fast balance reads without breaking 3NF. I accepted one controlled denormalised balance column and routed every change through the transactional transfer script, documenting the rule so later changes cannot bypass it.
- Making Q3 fast: I initially assumed a prefix filter on reference would use a plain B-tree index the way an equality filter does. I ran EXPLAIN several times before I trusted the bitmap index plan, and saved before-and-after screenshots to prove the improvement.
- Linking two stores without creating two ledgers: I had to resist putting monetary amounts into MongoDB. I kept customer_id and reference as the only join keys and left every debit and credit in PostgreSQL.
- Reproducible evidence: port and credential mismatches broke my early runs until I centralised start-up in one init script and a Docker Compose file. Now the whole evidence bundle regenerates with one command.
- Honest scope for cloud high availability: I could not justify the cost of running Multi-AZ RDS and Atlas for coursework, so I separated what I validated locally from what I researched and diagrammed as a production target.

Lessons learned

Polyglot persistence works when there is a hard line: strong consistency wherever money moves, and eventual consistency only where the business can accept lag. ER modelling, index tuning and transaction design reinforce each other, so I stopped treating them as isolated portfolio tasks. Automating evidence generation also protected me from silent regressions before packaging the submission.

6. REFERENCES

- Amazon Web Services, Inc. (2026). Amazon RDS user guide. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/ (accessed 13 November 2026).
- Federal Republic of Nigeria. (2023). Nigeria Data Protection Act 2023. National Assembly of the Federal Republic of Nigeria.

- Miva Open University. (2026). MIT 8103 Advanced Database Systems: Continuous assessment brief (September cohort). Course materials on the learning management system.
- MongoDB, Inc. (2026). MongoDB documentation. https://www.mongodb.com/docs/ (accessed 13 November 2026).
- PostgreSQL Global Development Group. (2026). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ (accessed 13 November 2026).
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill.

7. AI USE DECLARATION

I used Cursor AI to scaffold the project folders, suggest SQL and MongoDB patterns, render the Portfolio 5 architecture diagram and polish the wording of this document. I made the ValPay design decisions, ran the evidence scripts myself, and verified balances, rollback behaviour, EXPLAIN output and MongoDB query results against my local databases. I reviewed and corrected any wording that implied cloud resources I had not deployed, and I take responsibility for the accuracy of everything in the repository and submission.